

Dokumentacja Techniczna

Projektu: GameDeals Tracker



Przedmiot: Programowanie w języku Python (2025/2026)

Prowadzący: mgr inż. Wojciech Gałka

Autor: Michał Rajzer

Nr albumu: 134966

Rzeszów 2026 r

Spis treści

1. Opis Projektu.....	3
2. Schemat Bazy Danych.....	3
3. Moduł klient API.....	4
4. Logika serwera (Główna aplikacja app.py)	8
5. Warstwa frontendowa i interakcja	13
7. Możliwości rozwoju aplikacji.....	16
8. Bibliografia i źródła.....	16

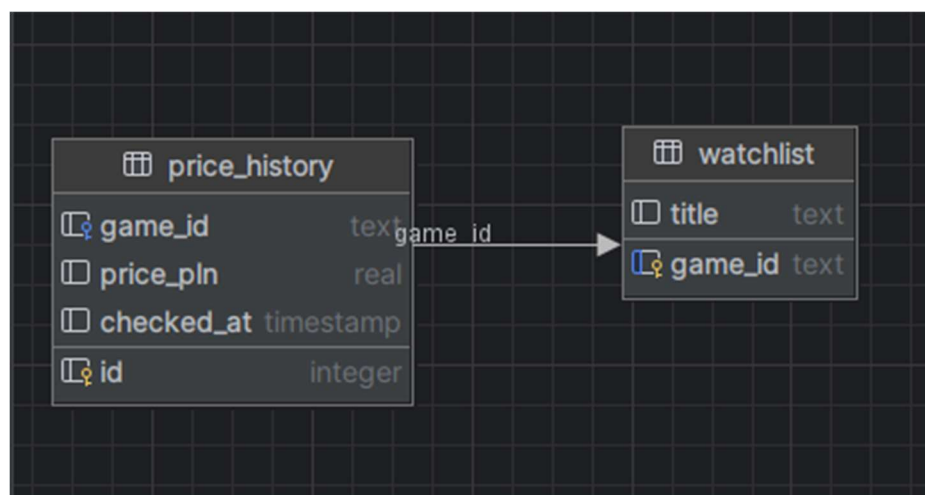
1. Opis Projektu

Aplikacja webowa napisana w Pythonie (Flask), która pozwala wyszukiwać promocje na gry komputerowe, przeliczać ich ceny na PLN oraz tworzyć własną listę obserwowanych tytułów wraz ze śledzeniem zmian cen w czasie.

Dane o ofertach pochodzą z publicznego API [CheapShark](#), a aktualny kurs wymiany dolara na złotówkę pobierany jest z API [Narodowego Banku Polskiego \(NBP\)](#).

2. Schemat Bazy Danych

Baza danych składa się z dwóch tabel powiązanych relacją jeden-do-wielu (1:N).



Rysunek 1: Schemat bazy

- Tabela watchlist - Przechowuje listę gier, które użytkownik zdecydował się obserwować.
- Tabela price_history - Przechowuje historię cen dodanych gier w celu określenia różnicy cenowej (od momentu rozpoczęcia śledzenia do aktualnej ceny).

Tabela `price_history` posiada zdefiniowany klucz obcy z klauzulą `ON DELETE CASCADE`. Powoduje to, że usunięcie gry z listy obserwowanych automatycznie wymazuje wszystkie powiązane z nią rekordy historii cen z bazy. W kodzie serwera przed każdą operacją wywoływane jest polecenie `PRAGMA foreign_keys = ON`, włączające egzekwowanie kluczy obcych w silniku SQLite.

3. Moduł klient API

Klasa `GameDataFetcher` stanowi warstwę komunikacyjną (klienta HTTP) pomiędzy główną aplikacją a zewnętrznymi interfejsami API. Odpowiada za bezpieczne wysyłanie zapytań, obsługę błędów sieciowych oraz formatowanie zwracanych danych.

Podczas inicjalizacji obiektu klasy `GameDataFetcher`, ustawiane są podstawowe konfiguracje (Konstruktor klasy):

- `base_url`: Główny adres (endpoint) darmowego API CheapShark (<https://www.cheapshark.com/api/1.0>).
- `store_mapping`: Słownik mapujący numeryczne identyfikatory sklepów z API na ich czytelne nazwy (np. 1 to "Steam", 25 to "Epic Games").

```
3 class GameDataFetcher: 2 usages  ⚡ Michał Rajzer
4     def __init__(self):  ⚡ Michał Rajzer
5         self.base_url = "https://www.cheapshark.com/api/1.0"
6         self.store_mapping = {
7             "1": "Steam",
8             "11": "GOG",
9             "25": "Epic Games"
10        }
```

Rysunek 2: konstruktor klasy

Metoda `search_deals(self, game_name)`

- **Zadanie:** Wyszukiwanie aktualnych ofert promocyjnych na podstawie przekazanego tytułu.
- **Działanie:** Buduje zapytanie GET do endpointu `/deals`. Posiada blok `try...except` chroniący przed błędami sieciowymi.
- **Zwraca:** Listę słowników (JSON) zawierających oferty lub `None` w przypadku błędu połączenia.

```
11
12  def search_deals(self, game_name): 1 usage  Michał Rajzer
13      url = f"{self.base_url}/deals"
14      params = {"title": game_name}
15      try:
16          response = requests.get(url, params=params)
17          response.raise_for_status()
18          return response.json()
19      except requests.exceptions.RequestException:
20          return None
21
```

Rysunek 3: Metoda `search_deals`

Metoda `get_usd_to_pln_rate(self)`

- **Zadanie:** Pobranie aktualnego średniego kursu wymiany USD na PLN z API Narodowego Banku Polskiego (NBP).
- **Działanie:** Wysyła zapytanie GET pod adres API NBP i parsuje strukturę JSON, aby wyodrębnić wartość kursu średniego (`mid`). Posiada obsługę wyjątków `try...except` z mechanizmem awaryjnym (`fallback`), który zwraca domyślną wartość 4.0, jeśli serwery NBP są niedostępne.
- **Zwraca:** Liczbę zmiennoprzecinkową będącą kursem dolara (np. 3.95) lub wartość zapasową 4.0 w przypadku błędu.

```

20
21 def get_usd_to_pln_rate(self): 3 usages  ⌵ Michał Rajzer
22     url = "http://api.nbp.pl/api/exchangerates/rates/a/usd/?format=json"
23     try:
24         response = requests.get(url)
25         response.raise_for_status()
26         return response.json()['rates'][0]['mid']
27     except requests.exceptions.RequestException:
28         return 4.0
29

```

Rysunek 4: Metoda `get_usd_to_pln_rate`

Metoda `get_game_by_id(self, game_id)`

- **Zadanie:** Pobranie szczegółowych informacji archiwalnych o konkretnej grze na podstawie unikalnego identyfikatora.
- **Działanie:** Odpytuje endpoint `/games` z parametrem `id`. Służy do pobrania historycznego minimum cenowego danej gry (pole `cheapestPriceEver`). Posiada blok `try...except` przechwytyjący błędy sieciowe.
- **Zwraca:** Słownik (JSON) ze szczegółowymi informacjami o grze lub `None` w przypadku błędu połączenia.

```

29
30 def get_game_by_id(self, game_id): 2 usages  ⌵ Michał Rajzer
31     url = f"{self.base_url}/games"
32     params = {"id": game_id}
33     try:
34         response = requests.get(url, params=params)
35         response.raise_for_status()
36         return response.json()
37     except requests.exceptions.RequestException:
38         return None
39

```

Rysunek 5: Metoda `get_game_by_id`

Metoda `get_games_by_ids(self, game_ids)`

- **Zadanie:** Zbiorcze pobranie danych dla wielu gier jednocześnie (Batch Request) w celu optymalizacji wydajności.
- **Działanie:** Formatuje listę identyfikatorów do postaci jednego tekstu rozdzielonego przecinkami i wysyła pojedyncze zapytanie do endpointu `/games`. Zapobiega to wielokrotnemu odpytywaniu API w pętli (problem N+1). Posiada pełne zabezpieczenie przed błędami połączenia.
- **Zwraca:** Słownik (JSON) z danymi gier (gdzie kluczem jest ID gry) lub pusty słownik `{}` w przypadku błędu bądź pustej listy wejściowej.

```
39
40  def get_games_by_ids(self, game_ids): 1 usage  Ⓜ Michał Rajzer
41  if not game_ids:
42      return {}
43  url = f"{self.base_url}/games"
44  params = {"ids": ",".join(game_ids)}
45  try:
46      response = requests.get(url, params=params)
47      response.raise_for_status()
48      return response.json()
49  except requests.exceptions.RequestException:
50      return {}
51
```

Rysunek 6: Metoda `get_games_by_ids`

Metoda `get_hot_deals(self)`

- **Zadanie:** Pobranie aktualnych, najpopularniejszych i najbardziej opłacalnych promocji cenowych na rynku.
- **Działanie:** Odpytuje endpoint `/deals` z nałożonymi filtrami: sortowanie według atrakcyjności (Deal Rating), status na promocji (`onSale=1`) oraz ograniczenie wyników do sklepów Steam, GOG i Epic Games. Zabezpieczona blokiem `try...except`.
- **Zwraca:** Listę słowników (JSON) zawierających najlepsze promocje lub `None` w przypadku błędu połączenia.

```

51
52     def get_hot_deals(self): 1 usage  Ⓜ Michał Rajzer
53         url = f"{self.base_url}/deals"
54         params = {"sortBy": "Deal Rating", "onSale": 1, "storeID": "1,11,25"}
55         try:
56             response = requests.get(url, params=params)
57             response.raise_for_status()
58             return response.json()
59         except requests.exceptions.RequestException:
60             return None

```

Rysunek 7: Metoda `get_hot_deals`

4. Logika serwera (Główna aplikacja `app.py`)

Plik `app.py` stanowi główny kontroler aplikacji. Konfiguruje on aplikację Flask, zarządza połączeniami z SQLite oraz definiuje trasy (routing) i operacje logiczne na danych.

```

9
10     def init_db(): 1 usage  Ⓜ Michał Rajzer
11         with sqlite3.connect('tracker.db') as conn:
12             c = conn.cursor()
13             c.execute("PRAGMA foreign_keys = ON")
14             c.execute('''CREATE TABLE IF NOT EXISTS watchlist
15                 (
16                     game_id TEXT PRIMARY KEY,
17                     title TEXT
18                 )''')
19             c.execute('''CREATE TABLE IF NOT EXISTS price_history
20                 (
21                     id INTEGER PRIMARY KEY AUTOINCREMENT,
22                     game_id TEXT,
23                     price_pln REAL,
24                     checked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
25                     FOREIGN KEY(game_id) REFERENCES watchlist(game_id) ON DELETE CASCADE
26                 )''')
27
28

```

Rysunek 8: Inicjalizacja bazy

Funkcja init_db()

- **Zadanie:** Inicjalizacja struktury bazy danych.
- **Działanie:** Nawiązuje połączenie z lokalnym plikiem bazy danych tracker.db, włącza sprawdzanie kluczy obcych (PRAGMA foreign_keys = ON) i tworzy tabele watchlist oraz price_history, jeśli one jeszcze nie istnieją. Definiuje kaskadowe usuwanie danych (ON DELETE CASCADE) dla powiązania klucza obcego.
- **Zwraca:** Brak wartości zwracanej (None).

Trasa / (Metoda index())

- **Zadanie:** Obsługa strony głównej aplikacji, wyszukiwania ofert oraz generowania wykresu porównawczego dla najlepszej oferty.
- **Działanie:** Obsługuje żądania GET i POST. Dla metody GET pobiera listę aktualnie śledzonych identyfikatorów gier z bazy, aby poprawnie oznaczyć status przycisków na froncie. Dla metody POST filtruje wyniki z CheapShark API na podstawie wybranych sklepów, konwertuje ceny w USD na PLN, sortuje je według wybranego kryterium i dla najkorzystniejszej oferty wyszukuje historyczne minimum, po czym przygotowuje dane wykresu Chart.js w formacie JSON.
- **Zwraca:** Wyrenderowany szablon HTML strony głównej (index.html) wraz z listą ofert, błędami, danymi wykresu oraz statusem śledzenia gier.

```

32  @app.route(rule: "/", methods=["GET", "POST"])  Michał Rajzer
33  def index():
34      deals = None
35      error = None
36      search_query = ""
37      chart_labels = ""
38      chart_data = ""
39      best_deal_title = ""
40      exchange_rate = api_fetcher.get_usd_to_pln_rate()
41
42      tracked_ids = []
43      try:
44          with sqlite3.connect('tracker.db') as conn:
45              c = conn.cursor()
46              c.execute("SELECT game_id FROM watchlist")
47              tracked_ids = [row[0] for row in c.fetchall()]
48      except Exception:
49          pass
50

```

Rysunek 9: Metoda index

Trasa /api/historical_low/<game_id> (Metoda historical_low(game_id))

- Zadanie: Obsługa asynchronicznego żądania (AJAX) pobierania najniższej historycznej ceny gry.
- Działanie: Pobiera szczegółowe dane gry na podstawie przekazanego identyfikatora i wyodrębnia z nich pole cheapestPriceEver.
- Zwraca: Odpowiedź w formacie JSON zawierającą wartość ceny w USD (price_usd) lub wartość None, jeśli dane są niedostępne.

```

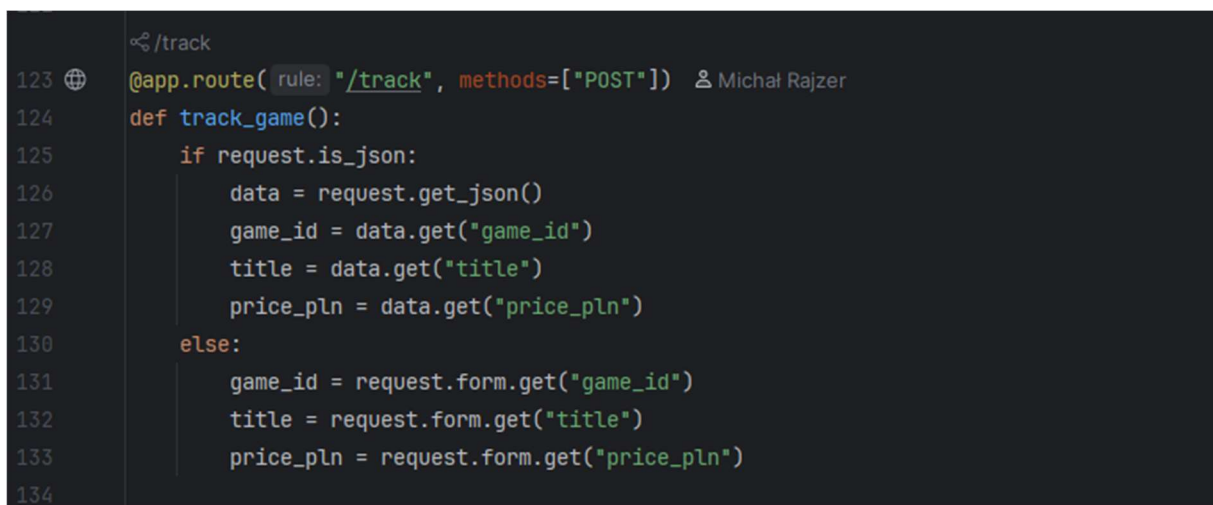
114
115  @app.route("/api/historical_low/<game_id>")  Michał Rajzer
116  def historical_low(game_id):
117      game_details = api_fetcher.get_game_by_id(game_id)
118      if game_details and "cheapestPriceEver" in game_details:
119          return jsonify({"price_usd": float(game_details["cheapestPriceEver"].get("price", 0))})
120      return jsonify({"price_usd": None})
121
122

```

Rysunek 10: Metoda historical_low

Trasa /track (Metoda track_game())

- **Zadanie:** Dodanie wybranej gry do listy obserwowanych oraz rejestracja jej ceny początkowej.
- **Działanie:** Obsługuje żądania przesłane w formacie JSON (AJAX) oraz w formacie formularza. Zapisuje identyfikator i tytuł gry w tabeli watchlist (używając INSERT OR IGNORE), a jeśli podano cenę, dodaje odpowiedni wpis z aktualną ceną do tabeli price_history w ramach transakcji SQLite.
- **Zwraca:** Odpowiedź w formacie JSON potwierdzającą sukces operacji lub informującą o błędzie (kod 400 lub 500).



```
123  @app.route(rule: "/track", methods=["POST"])  Michał Rajzer
124  def track_game():
125      if request.is_json:
126          data = request.get_json()
127          game_id = data.get("game_id")
128          title = data.get("title")
129          price_pln = data.get("price_pln")
130      else:
131          game_id = request.form.get("game_id")
132          title = request.form.get("title")
133          price_pln = request.form.get("price_pln")
134
```

Rysunek 11: Metoda track_game

Trasa /untrack (Metoda untrack_game())

- **Zadanie:** Usunięcie gry z listy obserwowanych.
- **Działanie:** Pobiera identyfikator gry z żądania JSON lub formularza, łączy się z bazą i wykonuje zapytanie DELETE na tabeli watchlist. Dzięki włączonej fladze foreign_keys oraz klauzuli ON DELETE CASCADE, usunięcie gry z watchlisty automatycznie kasuje wszystkie powiązane rekordy historii cen w tabeli price_history.
- **Zwraca:** Odpowiedź w formacie JSON potwierdzającą usunięcie gry lub informującą o błędzie (kod 400 lub 500).

```

149 </untrack
150 @app.route(rule: "/untrack", methods=["POST"]) & Michal Rajzer
151 def untrack_game():
152     if request.is_json:
153         data = request.get_json()
154         game_id = data.get("game_id")
155     else:
156         game_id = request.form.get("game_id")
157
158     if game_id:
159         try:
160             with sqlite3.connect('tracker.db') as conn:
161                 c = conn.cursor()
162                 c.execute("PRAGMA foreign_keys = ON")
163                 c.execute(sql: "DELETE FROM watchlist WHERE game_id = ?", parameters: (game_id,))
164                 return jsonify({"success": True, "message": "Gra usunięta z obserwowanych.", "action": "untrack", "game_id": game_id})
165             except Exception as e:
166                 return jsonify({"success": False, "error": str(e)}), 500
167
168     return jsonify({"success": False, "error": "Brak game_id"}), 400
169

```

Rysunek 12: Metoda untrack_game

Trasa /watchlist (Metoda watchlist())

- **Zadanie:** Obsługa strony listy obserwowanych gier, wyliczanie statystyk oraz różnic cenowych.
- **Działanie:** Pobiera z bazy listę śledzonych gier i dla każdego tytułu odczytuje najstarszą zarejestrowaną cenę (SELECT price_pln ... ORDER BY checked_at ASC LIMIT 1). Następnie zbiorczo odpytuje CheapShark API o aktualne ceny gier. Oblicza różnicę cenową (cena rynkowa minus cena początkowa) oraz wylicza globalne statystyki (łączna wartość standardowa i promocyjna, suma oszczędności, średni rabat, największa obniżka).
- **Zwraca:** Wyrenderowany szablon HTML listy obserwowanych (watchlist.html) wraz ze statystykami i listą gier zawierającą wskaźniki zmian cen.

```

1 </watchlist
2 @app.route("/watchlist") & Michal Rajzer
3 def watchlist():
4     tracked_games = []
5     tracked_prices = {}
6     try:
7         with sqlite3.connect('tracker.db') as conn:
8             c = conn.cursor()
9             c.execute("SELECT game_id, title FROM watchlist")
10            tracked_games = c.fetchall()
11            for game_id, _ in tracked_games:
12                c.execute(sql: "SELECT price_pln FROM price_history WHERE game_id = ? ORDER BY checked_at ASC LIMIT 1", parameters: (game_id,))
13                row = c.fetchone()
14                if row:
15                    tracked_prices[game_id] = row[0]
16            except Exception:
17                pass

```

Rysunek 13: Metoda watchlist

Trasa /top_deals (Metoda top_deals())

- Zadanie: Prezentacja 12 najgorętszych ofert z rynku na osobnej podstronie.
- Działanie: Odpytuje API CheapShark o najlepsze oferty, usuwa duplikaty gier o tym samym tytule, przelicza ceny na PLN według średniego kursu NBP, pobiera status śledzenia gier z bazy i ogranicza listę do pierwszych 12 pozycji.
- Zwraca: Wyrenderowany szablon HTML gorących okazji (top_deals.html) wraz z listą ofert oraz listą śledzonych ID.

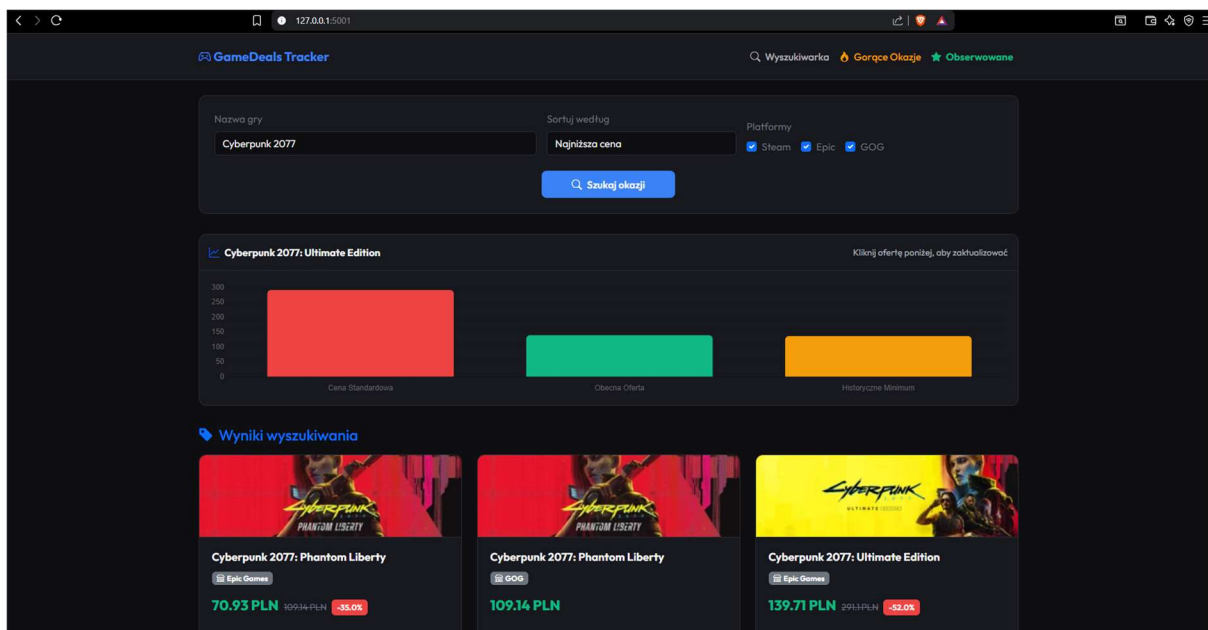
```
243
244 @app.route("/top_deals")  Michał Rajzer
245 def top_deals():
246     exchange_rate = api_fetcher.get_usd_to_pln_rate()
247     raw_deals = api_fetcher.get_hot_deals()
248     hot_deals = []
249     seen_titles = set()
250
251     tracked_ids = []
252     try:
253         with sqlite3.connect('tracker.db') as conn:
254             c = conn.cursor()
255             c.execute("SELECT game_id FROM watchlist")
256             tracked_ids = [row[0] for row in c.fetchall()]
257     except Exception:
258         pass
259
```

Rysunek 14: Metoda top_deals

5. Warstwa frontendowa i interakcja

Strona Główna / Wyszukiwarka

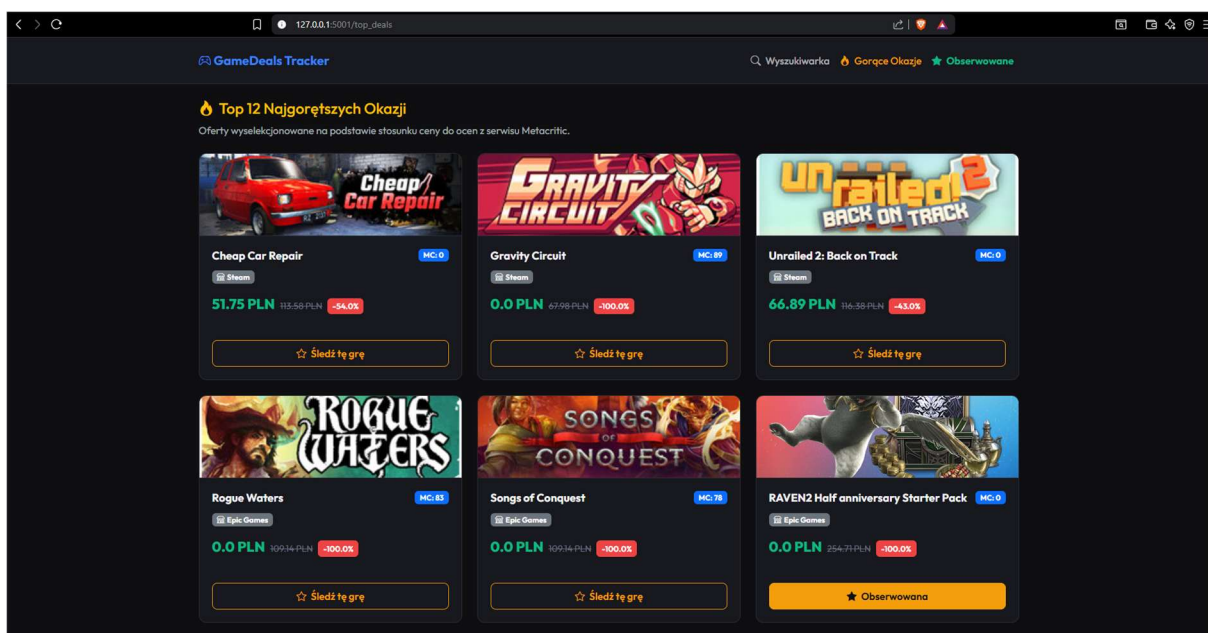
Użytkownik wpisuje nazwę gry, wybiera sklepy (Steam, GOG, Epic Games Store) oraz sposób sortowania, a następnie klika przycisk wyszukiwania. W wynikach pojawiają się karty gier z cenami i okładkami. Kliknięcie na dowolną kartę automatycznie aktualizuje wykres słupkowy u góry, porównując cenę standardową, promocyjną oraz historyczne minimum w PLN. Użytkownik może kliknąć przycisk gwiazdki, co asynchronicznie (bez odświeżania strony) dodaje lub usuwa grę z bazy danych.



Rysunek 15: Wyszukiwarka okazji

Gorące Okazje

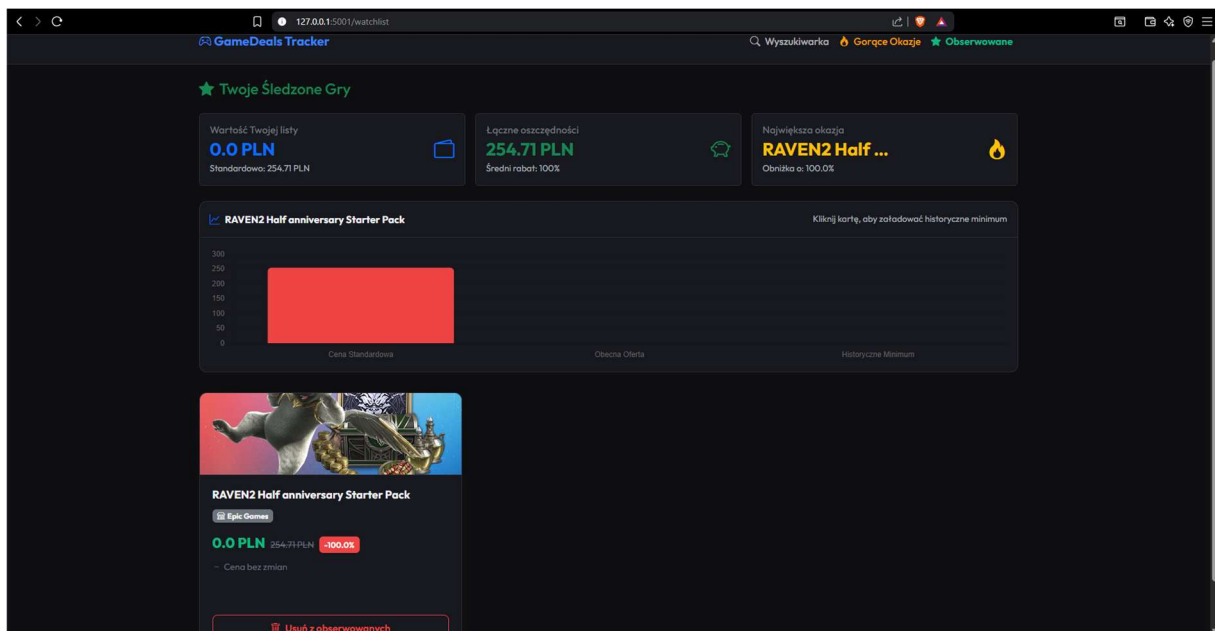
Po przejściu do tej zakładki system automatycznie pobiera z API i wyświetla 12 aktualnie najkorzystniejszych promocji na rynku. Każda karta zawiera logo sklepu, aktualną cenę w PLN, procent zniżki oraz ocenę gry z serwisu Metacritic. Podobnie jak na stronie głównej, użytkownik może bezpośrednio stąd zacząć obserwować grę, klikając przycisk gwiazdki, co wywoła zapytanie w tle.



Rysunek 16: najgorętsze okazje

Lista Obserwowanych

Strona wyświetla zestawienie gier śledzonych przez użytkownika. U góry znajduje się panel statystyk, który sumuje ceny standardowe, promocyjne oraz wylicza łączną kwotę zaoszczędzonych pieniędzy. Każda karta gry posiada unikalny wskaźnik pokazujący zmianę ceny od momentu rozpoczęcia śledzenia (spadek ceny wyświetla się na zielono, wzrost na czerwono, a brak zmian na szaro). Kliknięcie "Usuń z obserwowanych" wywołuje zapytanie AJAX – karta gry płynnie znika, a statystyki u góry natychmiast się przeliczają.



Rysunek 17: Widok obserwowanych gier ze statystykami i historią cen

7. Możliwości rozwoju aplikacji

Aplikacja GameDeals Tracker została zaprojektowana w sposób modułowy, co ułatwia jej ewentualną rozbudowę w przyszłości. Główne kierunki rozwoju systemu to:

- System rejestracji i logowania (Autoryzacja) – Wdrożenie systemu kont użytkowników (np. przy użyciu Flask-Login), aby każdy użytkownik posiadał własną, spersonalizowaną listę obserwowanych gier, chronioną hasłem.
- Automatyczne powiadomienia e-mail – Integracja z biblioteką Flask-Mail w celu wysyłania powiadomień e-mail bezpośrednio na skrzynkę użytkownika w momencie, gdy cena śledzonej gry spadnie poniżej zdefiniowanego przez niego progu.
- Cykliczny zapis cen w tle (Wykresy zmian cen w czasie) – Uruchomienie skryptu który raz na dobę automatycznie odpyta API CheapShark i zapisze nowe punkty cenowe w bazie. Pozwoli to na zastąpienie wykresu słupkowego liniowym, obrazującym pełną historię wahań ceny gry na przestrzeni miesięcy.
- Wsparcie dla innych walut (Wielowalutowość) – Wykorzystanie pełnej tabeli A kursów średnich NBP (np. pobieranie kursów EUR, GBP), co dałoby użytkownikowi możliwość wyboru wyświetlanej waluty na żywo w interfejsie.

8. Bibliografia i źródła

1. [Dokumentacja CheapShark API](#) – Oficjalne wytyczne API agregatora promocji cenowych
2. [NBP Web Services API](#) – Publiczne serwisy API Narodowego Banku Polskiego do pobierania aktualnych kursów walut
3. [Flask Documentation](#) – Oficjalna dokumentacja ramki webowej Flask
4. [Chart.js Documentation](#) – Dokumentacja biblioteki JavaScript służącej do generowania wykresów
5. [Bootstrap Framework](#) – Oficjalna dokumentacja biblioteki stylów i komponentów RWD
6. [Repozytorium GitHub](#) – Kod źródłowy projektu GameDeals Tracker:
https://github.com/MR134966/Games_Tracker.git